

bridges.

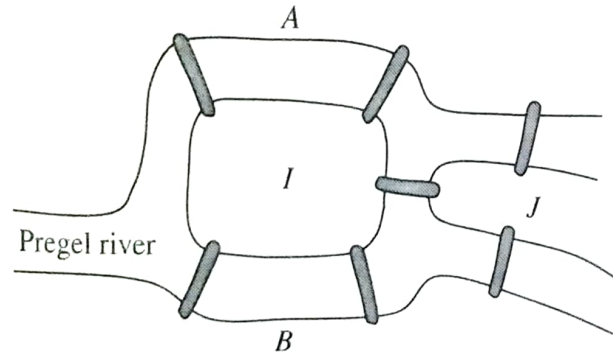


Fig. 13.1 Bridges of Königsberg

The Königsberg bridge problem was to find whether it was possible to walk through the town in such a way as to traverse every bridge exactly once. In 1736, Leonhard Euler, a Swiss mathematician, came out with the solution in terms of graph theory. He presented the problem in a simple way by representing the landmasses by dots and the bridges by lines that

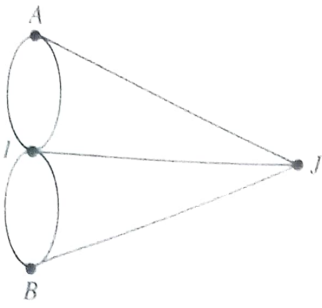


Fig. 13.2 Graph of Königsberg bridge problem

connected those landmasses. Figure 13.2 represents the Königsberg bridge problem as a graph. Euler proved that it was not possible to walk through the town by traversing through the seven bridges, crossing each bridge exactly once. He also explained why it was not possible. He introduced the concept of *degree of a node*, the number of edges touching a node, and proposed that any given graph can be traversed with each edge traversed exactly once if and only if it had zero or exactly two nodes of odd degree.

Graph theory deals with the study of graphs and their various properties. It has wide applications, as many real-life problems can be modelled through graphs. Communication network, data organization, link structures of web pages, job assignment, electrical network, and so on can easily be understood through graphs. In this chapter, we shall study graph as a formal mathematical structure. We shall also define various terms used in graph theory and discuss different properties of graphs.

13.2 GRAPH AND ITS RELATED DEFINITIONS

A graph $G(V, E)$ consists of a set of vertices denoted by V or $V(G)$ and a set of edges denoted by E or $E(G)$. Every edge is associated with an unordered pair of vertices.

Figure 13.3 represents a graph $G(V, E)$ having seven vertices and eight edges. The set of vertices and edges are defined as follows:

$$V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7\} \quad \text{and} \quad E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8\}$$

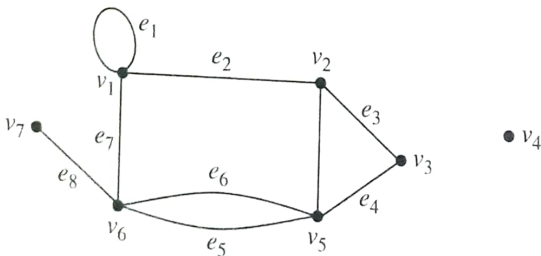


Fig. 13.3 Graph with self-loop and parallel edges

An edge is also written as an unordered pair of vertices. For example, in Fig. 13.3, e_2 can be written as (v_1, v_2) or sometimes $v_1 v_2$.

Some definitions useful in further study of graphs are provided here.

Order and Size of a Graph

The number of vertices in a graph or the cardinality of the set $V(G)$ is said to be the order of the graph, and the number of edges in the graph $E(G)$ is said to be the size of the graph. Graphs are finite or infinite according to their order and size; unless otherwise stated, the graphs we consider are all finite.

Self-loop and Parallel Edges

An edge starting and ending in the same vertex is called a self-loop. In Fig. 13.3 the edge e_1 forms a self-loop. Edges associated with the same pair of

vertices are called parallel edges. In Fig. 13.3, the edges e_5 and e_6 are parallel edges.

Adjacent Vertices and Edges

Two vertices are said to be adjacent if there is an edge between them. Two non-parallel edges are called adjacent if they are incident on a common vertex. In Fig. 13.3, the vertices v_1 and v_2 are adjacent vertices and the edges e_3 and e_4 are adjacent edges.

An edge is called incident to a vertex if the vertex is the end vertex of the edge. In Fig. 13.3, the edge e_3 is incident on the vertices v_2 and v_3 .

Degree of a Vertex

The degree of a vertex v is the number of edges incident on the vertex. It is a positive number and is denoted by $\deg(v)$. A loop is counted twice for the calculation of degree of a vertex. In Fig. 13.3, the degrees of some of the vertices are as follows:

$$\deg(v_1) = 4, \deg(v_2) = 3, \deg(v_3) = 2, \deg(v_5) = 4$$

In a graph $G(V, E)$, the minimum and maximum degrees of a vertex are denoted by $\delta(G)$ and $\Delta(G)$, respectively.

$$\delta(G) = \text{Min}(\deg(v) : v \in G(v))$$

$$\Delta(G) = \text{Max}(\deg(v) : v \in G(v))$$

Isolated Vertex and Pendent Vertex

A vertex is called an isolated vertex if no edge is incident on the vertex. The degree of an isolated vertex is zero. In Fig. 13.3, the vertex v_4 is an isolated vertex. A vertex is called a pendent vertex if the degree of the vertex is one. In Fig. 13.3, the vertex v_7 is a pendent vertex.

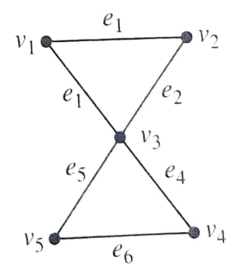


Fig. 13.4 Graph for Example 13.1

EXAMPLE 13.1 Define formally the graph given in Fig. 13.4, that is, the set of vertices, the set of edges, and the degree of each vertex.

Solution: The graph $G(V, E)$ can be defined as follows:
 $V = \{v_1, v_2, v_3, v_4, v_5\}$; $E = \{e_1, e_2, e_3, e_4, e_5, e_6\}$; and $\deg(v_1) = 2$, $\deg(v_2) = 2$, $\deg(v_3) = 4$, $\deg(v_4) = 2$, and $\deg(v_5) = 2$

EXAMPLE 13.2 Define formally the graph given in Fig. 13.5, that is, the set of vertices, the set of edges, and the degree of each vertex.

Solution: The graph $G(V, E)$ can be defined as follows:
 $V = \{v_1, v_2, v_3, v_4, v_5\}$; $E = \{(v_1, v_2), (v_2, v_2), (v_1, v_3), (v_3, v_4)\}$; and $\deg(v_1) = 2$, $\deg(v_2) = 3$, $\deg(v_3) = 2$, $\deg(v_4) = 1$, and $\deg(v_5) = 0$

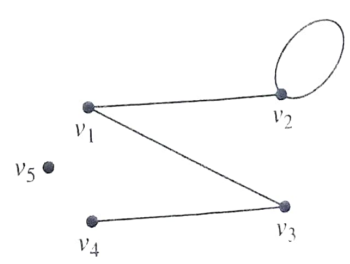


Fig. 13.5 Graph for Example 13.2

THEOREM 13.1 Let $G(V, E)$ be a graph having n vertices e edges. Then

show that $\sum_{i=1}^n \deg(v_i) = 2e$.

Proof: Consider a graph of n vertices $v_1, v_2, v_3, \dots, v_n$ and e edges. Every edge is incident on two vertices; hence, every edge is counted twice for the calculation of the total degree of all vertices. This fact leads to the conclusion that the sum of degrees of all vertices is twice the number of edges in the graph. That is,

$$\sum_{i=1}^n \deg(v_i) = 2e \quad (13.1)$$

COROLLARY 13.2 Prove that in a graph G the number of vertices having an odd degree is always even.

Proof: Let us consider a graph $G(V, E)$ having n vertices $V = \{v_1, v_2, v_3, \dots, v_n\}$ and e edges. Consider a partition of the set V into two disjoint sets V_E and V_o , where V_E is the set of vertices having even degrees and V_o is the set of vertices having odd degrees. Thus, we have

$$V_E \cup V_o = V \quad \text{and} \quad V_E \cap V_o = \phi$$

which implies that $\sum_{i=1}^n \deg(v_i) = \sum_{v_i \in V_E} \deg(v_i) + \sum_{v_i \in V_o} \deg(v_i)$.

$$2e = \sum_{v_i \in V_E} \deg(v_i) + \sum_{v_i \in V_o} \deg(v_i) \quad (\text{since } \sum_{i=1}^n \deg(v_i) = 2e)$$

$$2e = \text{An even number less than } 2e + \sum_{v_i \in V_o} \deg(v_i)$$

$$\sum_{v_i \in V_o} \deg(v_i) = 2e - \text{an even number less than } 2e$$

$$= \text{An even number}$$

Since all vertices in V_o have odd degrees, the sum of odd degrees of all vertices would be an even number if there are even vertices in the set V_o (as odd numbers added even times result in an even number).

COROLLARY 13.3 Let $G(V, E)$ be a graph having n vertices e edges. Then show that $\delta \leq \frac{2e}{n} \leq \Delta$.

Proof: We know that $\sum_{i=1}^n \deg(v_i) = 2e$. Since δ and Δ are the minimum and maximum degrees, respectively, of a vertex in a graph, replacing the degree of each vertex by δ and Δ , we have $n\delta \leq 2e \leq n\Delta$. This implies that $\delta \leq \frac{2e}{n} \leq \Delta$.

EXAMPLE 13.3

A graph has 12 edges, two vertices of degree 3, two vertices of degree 4, and other vertices of degree 5. Find the number of vertices in the graph.

Solution: Let there be n vertices in the graph.

Sum of degrees of all vertices $= 2e$

$$2 \times 3 + 2 \times 4 + (n - 4) \times 5 = 24$$

$$(n - 4) = 2$$

$$n = 6$$

13.3 DIFFERENT TYPES OF GRAPHS

In this section, we shall discuss the different types of graphs.

13.3.1 Simple Graph

A graph without self-loops and parallel edges is called a simple graph (Fig. 13.6).

A graph is called finite if it has a finite number of edges and a finite number of vertices; otherwise, it is an infinite graph.

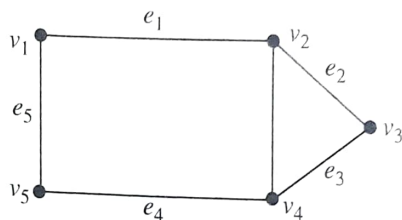


Fig. 13.6 Simple graph

13.3.2 Multigraph, Trivial Graph, and Null Graph

A graph having some parallel edges is called a multigraph. A graph is called a trivial graph if it has one vertex and no edges. A graph having finite vertices is called a null graph if it has no edges (Fig. 13.7).

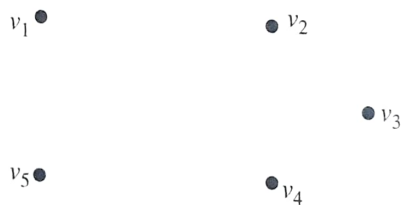


Fig. 13.7 Null graph

13.3.3 Complete Graph

A simple graph is said to be a complete graph if there exists an edge between every pair of vertices. A complete graph having n vertices is denoted by K_n . Figure 13.8 shows examples of complete graphs.

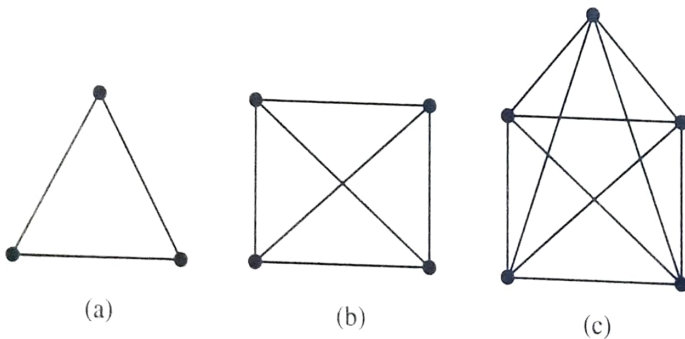


Fig. 13.8 Complete graphs (a) K_3 (b) K_4 (c) K_5

13.3.4 Regular Graph

A simple graph is said to be regular if the degree of each vertex is the same. If the degree of each vertex of a regular graph equals r , the graph is said to be r -regular. Figure 13.9 shows 2-regular and 3-regular graphs.

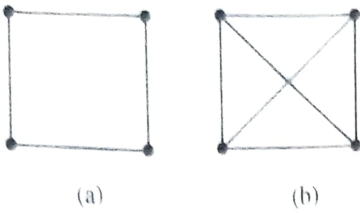


Fig. 13.9 Regular graphs
(a) 2-regular (b) 3-regular

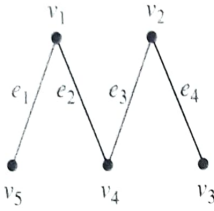


Fig. 13.10 Bipartite graph

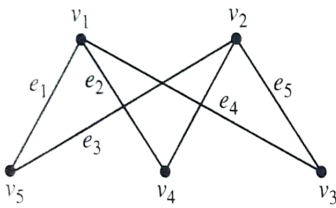


Fig. 13.11 Complete bipartite graph

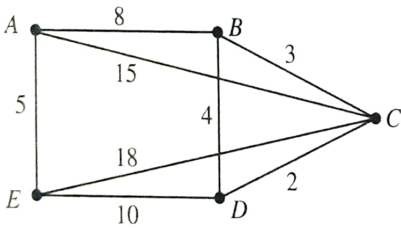


Fig. 13.12 Weighted graph showing traffic density of different routes in a city

From the aforementioned discussion, it can be observed that every complete graph is a regular graph, but every regular graph is not necessarily a complete graph.

13.3.5 Bipartite Graph

A graph $G(V, E)$ is said to be a bipartite graph if there exists a partition of the set $V(G)$ into two disjoint sets $V_1(G)$ and $V_2(G)$ such that each edge of the graph has its one end in $V_1(G)$ and the other end in $V_2(G)$.

The graph shown in Fig. 13.10 is a bipartite graph. In this graph, the set $V(G)$ is partitioned into two disjoint sets $V_1(G)$ and $V_2(G)$, where

$$V_1(G) = \{v_1, v_2\} \text{ and } V_2(G) = \{v_3, v_4, v_5\}$$

A bipartite graph is called complete bipartite if each vertex of $V_1(G)$ is joined to each vertex of $V_2(G)$ through an edge (Fig. 13.11).

13.3.6 Weighted Graph

A graph is called a weighted graph if all its edges have been assigned some positive real numbers (weights) to provide some additional information. For example, if we construct a graph of different places and the routes between two places in a metropolitan city, then the traffic density (average number of vehicles per minute moving out through the route) of the different routes can be shown by defining weights to different edges (Fig. 13.12).

EXAMPLE 13.4

Let G be a simple graph with n vertices. Show that the number of edges in the graph is $\frac{n(n-1)}{2}$ if the graph is a complete graph.

Solution: Since the graph is a complete graph, there exists an edge between every pair of vertices. An edge can be represented as an unordered pair of vertices. Therefore, the total number of edges can be calculated by counting the number of unordered pair of vertices that can be formed from n vertices. The number of such combinations is given by

$${}^nC_2 = \frac{n(n-1)}{2}.$$

EXAMPLE 13.5

Let e denote the number of edges in a complete bipartite graph $K_{m,n}$. Then show that $e(K_{m,n}) = mn$.

13.6.1 Walk

A walk in a graph is an alternating sequence of vertices and edges starting and ending in vertices such that every edge between two vertices in the sequence is incident on the two vertices. In a walk, a vertex can appear more than once but an edge cannot appear more than once. The first and last vertices of the walk are said to be the terminal vertices. The graphs in Figs 13.25(b) and (c) show two walks in the graph given in Fig. 13.25(a).

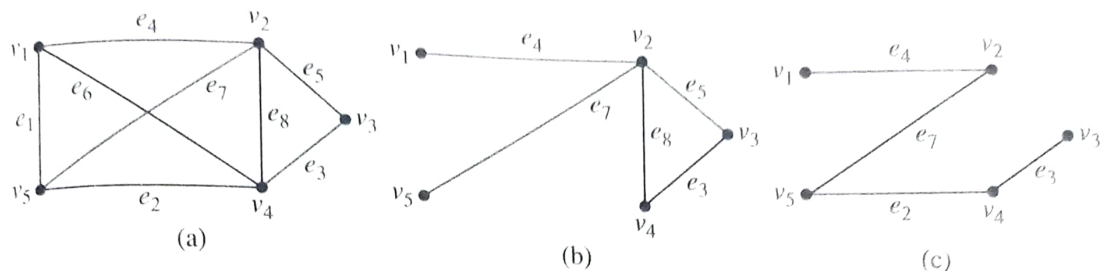


Fig. 13.25 Graph and its walks (a) $G(V, E)$ (b) Walk 1 $v_1e_4v_2e_8v_4e_3v_3e_5v_2e_7v_5$
(c) Walk 2 $v_1e_4v_2e_7v_5e_2v_4e_3v_3$

A walk that begins and ends at the same vertex is called a closed walk. If the terminal vertices are different, the walk is said to be open.

13.6.2 Path

An open non-intersecting walk is called a path. The term non-intersecting implies that a path cannot intersect itself. In other words, an open walk is called a path if all the vertices of the walk are distinct or no vertex appears more than once in the walk. A path is also called a simple path or an elementary path. In a path, the terminal vertices are of degree one and other intermediate vertices are of degree two. In Fig. 13.25, walk 2 is a path whereas walk 1 is not a path.

The length of a path is the number of edges in the path. Two vertices are said to be *reachable* from each other if there exists a path between them. In Fig. 13.25(a), the vertex v_1 is reachable from the vertex v_4 .

13.6.3 Circuit

A closed non-intersecting walk is called a circuit. A circuit is also called a cycle or a circular path. The degree of every vertex in a circuit is two. In Fig. 13.25(b), the part $v_2e_8v_4e_3v_3e_5v_2$ of walk 1 is a circuit.

In some graph theory literature, a walk is also defined with repetition of edges and *trail* is defined as a walk without repetition of edges, circuit as a closed trail, and *cycle* as a closed walk with distinct vertices except first and last vertex. However, keeping in mind the usage and simplification of the terms, we shall use these terms as given in our definition throughout the chapter.

13.7 CONNECTED GRAPH, DISCONNECTED GRAPH, AND COMPONENTS

A graph is said to be connected if every two vertices are reachable from each other (Fig. 13.26). Otherwise, the graph is said to be disconnected (Fig. 13.27).

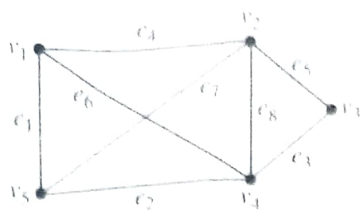


Fig. 13.26 Connected graph

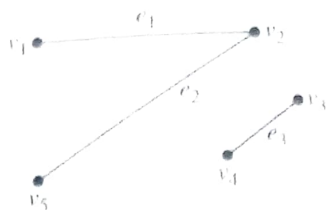


Fig. 13.27 Disconnected graph

Every disconnected graph can be partitioned into connected subgraphs and these connected subgraphs are called components. The disconnected graph given in Fig. 13.27 consists of two components.

THEOREM 13.4 A graph G is disconnected if and only if its vertex set V is partitioned into two non-empty, disjoint subsets V_1 and V_2 such that there exists no edge in G whose one end vertex is in V_1 and the other is in V_2 .

Proof: Let the graph G be disconnected. Every disconnected graph contains some components. Let V_1 and V_2 be the two components. Then $V_1 \cup V_2 = V$ and $V_1 \cap V_2 = \emptyset$. Thus, $\{V_1, V_2\}$ forms a partition of V such that there exists no edge in G whose one end vertex is in V_1 and the other is in V_2 .

Let the vertex set V be partitioned into two non-empty, disjoint subsets V_1 and V_2 such that there exists no edge in G whose one end vertex is in V_1 and the other is in V_2 . Since the vertices of the set V_1 are not connected by the vertices of the set V_2 , the graph is disconnected.

This proves the theorem.

THEOREM 13.5 If a graph G (connected or disconnected) has exactly two vertices of odd degree, there must be a path joining the two vertices.

Proof: Let the graph G be connected. Then there exists a path between each pair of vertices. Thus if the graph G is connected and has exactly two vertices of odd degree, there will be a path between the two vertices.

Let the graph G be disconnected and let it have exactly two vertices of odd degree. We know that every disconnected graph contains components, and a component is a subgraph of the graph G and hence forms a graph itself. Since the number of vertices of odd degree in a graph must be even and there are exactly two vertices of odd degree, the two vertices must be a part of one component. As every component is a connected graph, there exists a path between the two vertices.

This proves the theorem.

THEOREM 13.6 A disconnected simple graph G (without self-loops and parallel edges) with n vertices and k components can have at most $\frac{(n-k)(n-k+1)}{2}$ edges.

Proof: Let the k components be $G_1, G_2, \dots, G_k$ that contain $n_1, n_2, \dots, n_k$ number of vertices, respectively. Thus, $n_1 + n_2 + \dots + n_k = n$ or $\sum_{i=1}^k n_i = n$.

We know that every component is a connected graph and the maximum number of edges in a simple connected graph is given by $\frac{n(n-1)}{2}$. Let the maximum number of edges in the graph G be denoted by e . Then

$$\begin{aligned}
 e &= \frac{n_1(n_1-1)}{2} + \frac{n_2(n_2-1)}{2} + \dots + \frac{n_k(n_k-1)}{2} \\
 &= \sum_{i=1}^k \frac{n_i(n_i-1)}{2} \\
 &= \sum_{i=1}^k \frac{1}{2} (n_i^2 - n_i) \\
 &= \frac{1}{2} \sum_{i=1}^k n_i^2 - \frac{1}{2} \sum_{i=1}^k n_i \\
 &= \frac{1}{2} \sum_{i=1}^k n_i^2 - \frac{n}{2}
 \end{aligned} \tag{13.2}$$

We have $\sum_{i=1}^k (n_i - 1) = n - k$.

Squaring both sides, we get

$$\begin{aligned}
 \left(\sum_{i=1}^k (n_i - 1) \right)^2 &= (n - k)^2 \\
 \Rightarrow [(n_1 - 1) + (n_2 - 1) + \dots + (n_k - 1)]^2 &= n^2 + k^2 - 2nk \\
 \Rightarrow (n_1 - 1)^2 + (n_2 - 1)^2 + \dots + (n_k - 1)^2 + A &= n^2 + k^2 - 2nk
 \end{aligned}$$

($A = A$ non-negative term that is a sum of the products of two terms)

$$\begin{aligned}
 \Rightarrow \sum_{i=1}^k n_i^2 + k - 2 \sum_{i=1}^k n_i + A &= n^2 + k^2 - 2nk \\
 \Rightarrow \sum_{i=1}^k n_i^2 + k - 2n &\leq n^2 + k^2 - 2nk \\
 \Rightarrow \sum_{i=1}^k n_i^2 &\leq n^2 + k^2 - 2nk - k + 2n \\
 \Rightarrow \sum_{i=1}^k n_i^2 &\leq n^2 - (k-1)(2n-k)
 \end{aligned}$$

Substituting the value of $\sum_{i=1}^k n_i^2$ in Eq. (13.2), we get

$$\begin{aligned}
 e &\leq \frac{1}{2}n^2 - \frac{1}{2}(k-1)(2n-k) - \frac{n}{2} \\
 \Rightarrow e &\leq \frac{1}{2}[n-k+n^2+k^2-2nk] \\
 \Rightarrow e &\leq \frac{1}{2}[n-k+(n-k)^2] \\
 \Rightarrow e &\leq \frac{(n-k)(n-k+1)}{2}
 \end{aligned}$$

This proves the theorem.

Distance between Two Vertices

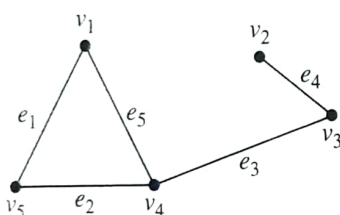


Fig. 13.28 Distance between two vertices

In a connected graph $G(V, E)$, the distance between two vertices v_i and v_j is the length of the shortest path between v_i and v_j . The distance between vertices v_i and v_j is denoted by $d(v_i, v_j)$. In Fig. 13.28, the distance between v_5 and v_3 is two.

Eccentricity, Center, and Diameter

The eccentricity of a vertex v , denoted by $E(v)$, is the distance from the vertex v to the vertex that is at the maximum distance from v .

$$E(v) = \text{Max} \{d(v, v_i), v_i \in G\}$$

A vertex having minimum eccentricity is called the center of the graph and the eccentricity of the center is called radius of the graph. In Fig. 13.28,

$$E(v_1) = 3, E(v_2) = 3, E(v_3) = 2, E(v_4) = 2, \text{ and } E(v_5) = 3$$

Thus, v_3 and v_4 are the two centers of the graph. The radius of the graph is two.

The diameter of a graph is the maximum distance of any two vertices of the graph and is denoted by $\text{dia}(G)$. In Fig. 13.28, the diameter of the graph is three.

13.8 HOMOMORPHISM AND ISOMORPHISM OF GRAPHS

Let $G(V, E)$ and $G'(V', E')$ be two graphs. A homomorphism from G to G' is a function $f: G \rightarrow G'$ that preserves edges; that is, if (u, v) is any edge of G and suppose u' and v' are the vertices of G' corresponding to the vertices u and v of G , then (u', v') is an edge of G' . Formally, homomorphism can be defined as follows:

A function $f: G \rightarrow G'$ is a homomorphism from G to G' if $f = (f_V, f_E)$, where $f_V: G(V) \rightarrow G'(V')$ and $f_E: G(E) \rightarrow G'(E')$ satisfy the following condition:

For an edge $(u, v) \in G$, $f_E(u, v) = (f_V(u), f_V(v))$.

For the graphs given in Fig. 13.29, there exists a homomorphism from the graph G to the graph G' . We can define a mapping $f: G \rightarrow G'$ such that $f_v(u_1) = v_1$, $f_v(u_2) = v_1$, and $f_v(u_3) = v_2$. Thus, $f_e(u_2, v_3) = (f_v(u_2), f_v(v_3)) = (v_1, v_2)$.

A homomorphism $f: G \rightarrow G'$ is called an *isomorphism* if both maps f_v and f_e are bijective. In this case, we say that G and G' are *isomorphic* to each other.

If the two graphs G and G' are isomorphic, then the number of vertices and number of edges in both the graphs are the same and there exists a homomorphism from G to G' and also from G' to G .

The two graphs given in Fig. 13.30 are isomorphic. There exists a one-one onto mapping $f: G \rightarrow G'$ such that

$$f(v_1) = v'_1, f(v_2) = v'_2, f(v_3) = v'_3,$$

$$f(v_4) = v'_4, \text{ and}$$

$$f(e_1) = e'_1, f(e_2) = e'_2, f(e_3) = e'_3$$

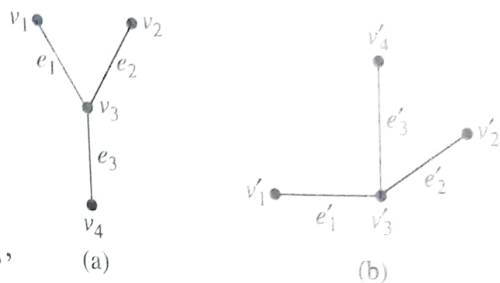


Fig. 13.30 Isomorphic graphs
(a) $G(V, E)$ (b) $G'(V', E')$

The two graphs given in Fig. 13.29 are not isomorphic to each other as the mapping f_v is not a bijection.

13.9 HOMEOMORPHIC GRAPHS

Let $G(V, E)$ be a graph and $e = (u, v)$, where e is an edge of the graph joining the two vertices u and v . An *edge subdivision* is the process of replacing an edge e by two edges (say e_1 and e_2) and a new vertex of degree two (say w) such that $e_1 = (u, w)$ and $e_2 = (w, v)$. A graph G' is called a subdivision of G if G' is obtained from G by a sequence of subdivisions of edges in G . Two graphs $G_1(V_1, E_1)$ and $G_2(V_2, E_2)$ are said to be *homeomorphic* if both the graphs are subdivisions of the same graph $G(V, E)$. The two graphs given in Fig. 13.31 are homeomorphic, since both the graphs are subdivisions of the complete graph of three vertices, that is, K_3 .

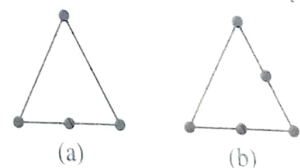


Fig. 13.31 Homeomorphic graphs
(a) $G(V, E)$
(b) $G'(V', E')$

For a graph $G(V, E)$ and $e = (u, v)$, an *elementary contraction* of G by the edge e is the process of deleting the edge e and merging the two vertices u and v to form a single vertex (say w) such that w is adjacent to those vertices that were adjacent to u and v in G after deleting any parallel edges that may appear due to the process. A graph G' is called a contraction of a graph G if G' can be obtained from G by a sequence of elementary contractions.

Let G and G' be two graphs. We will check (a) if G' is a subdivision of G , whether G will be a contraction of G' (b) if G is a contraction of G' , whether G' will be a subdivision of G . Let us consider the graphs given in Figs 13.32 and 13.33. In Fig. 13.32, G' is the contraction of G and G is a subdivision of G' .

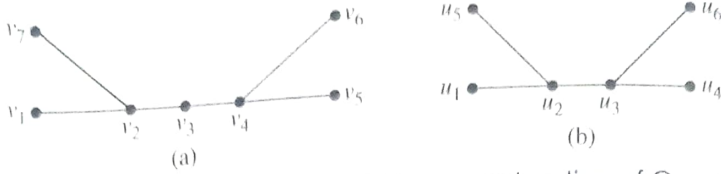


Fig. 13.32 Graphs G and G' where G' is the contraction of G and G is a subdivision of G' (a) Graph G (b) Graph G'

However, in Fig. 13.33, G' is the contraction of G but G is not a subdivision of G' . Thus, if the graph G' is a subdivision of the graph G , then G is a contraction of G' , but if G is a contraction of G' , then it is not necessary that G' is a subdivision of G .

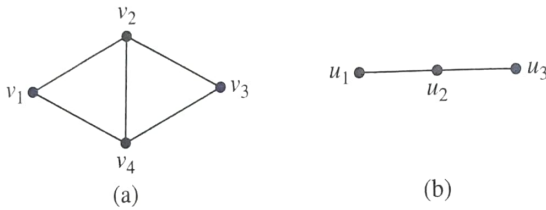


Fig. 13.33 Graphs G and G' where G' is the contraction of G but G is not a subdivision of G' (a) Graph G (b) Graph G'

13.10 EULER AND HAMILTONIAN GRAPHS

In this section we will describe the graphs in which it is possible to find closed walks that contain all the edges/vertices of the graph. These graphs have many practical applications in real life problems.

13.10.1 Euler Line and Euler Graph

The concept of Euler graph came from the question in what type of graph G it is possible to find a closed walk passing through every edge of G , which Euler has described in his paper dealing with the Königsberg bridge problem.

A closed walk that contains all edges of a graph is called an *Euler line*, and a graph that contains an Euler line is called an *Euler graph* (Fig. 13.34a). We know that a walk traces each edge exactly once and it is connected. Since an Euler graph contains all edges of a graph, it is always connected, and hence Euler graphs do not have isolated vertices. An open walk that includes all edges of a

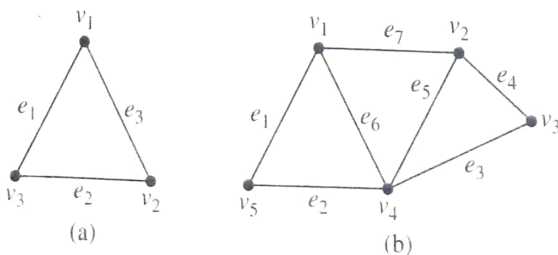


Fig. 13.34 Euler and unicursal graphs (a) Euler graph (b) Unicursal graph

graph is called a *unicursal line* or an open Euler line. A connected graph that has a unicursal line is called a *unicursal graph* (Fig. 13.34b). In Fig. 13.34(b), the open Euler line is $v_1e_1v_5e_2v_4e_6v_1e_7v_2e_4v_3e_3v_4e_5v_2$.

From the definition of a unicursal line, it is clear that by adding an edge between the initial and final vertices of a unicursal line, we will get an Euler line. Thus, a connected graph is unicursal if and only if it has exactly two vertices of odd degree.

13.10.2 Hamiltonian Path and Hamiltonian Circuit

A Hamiltonian circuit is a closed walk that traverses each vertex of a graph G exactly once except the starting vertex at which the walk also terminates. In the graph shown in Fig. 13.35, $v_1e_3v_3e_5v_5e_6v_4e_8v_2e_1v_1$ is a Hamiltonian circuit (shown in in bold lines)

Not every circuit in a graph is a Hamiltonian circuit. A circuit in a graph is said to be Hamiltonian if it includes all vertices of the graph. Hence, a Hamiltonian circuit in a graph of n vertices contains exactly n vertices and n edges.

It should be remembered that every connected graph need not contain a Hamiltonian circuit. There is no criterion or condition through which we can determine the existence of a Hamiltonian circuit in a graph.

A path obtained by removing one edge from a Hamiltonian circuit is called a *Hamiltonian path*. Thus, a Hamiltonian path contains all vertices of the graph and the length of the Hamiltonian path in a graph of n vertices is $n - 1$. Every graph that has a Hamiltonian circuit also has a Hamiltonian path but its converse is not true.

Self-loops and parallel edges cannot be included in a Hamiltonian circuit (path) as a Hamiltonian circuit (path) traverses each vertex exactly once. Therefore, in searching for the existence of a Hamiltonian circuit (path) in a given graph, the graph can be made a simple graph by removing all self-loops and parallel edges. Each member of a family of complete graphs having three or more vertices contains a Hamiltonian circuit.

A given graph may have more than one Hamiltonian circuit. As regards the presence of edge disjoint Hamiltonian circuits, the determination of the exact number of edge disjoint Hamiltonian circuits in a graph is also an unsolved problem. However, in a complete graph with odd number of vertices, the number of edge disjoint Hamiltonian circuits can be calculated.

13.10.3 Travelling Salesman Problem

The traveling salesman problem was studied in the 18th century by Sir William Rowan Hamilton, an Irish mathematician, and by Thomas Penyngton Kirkman, a British mathematician. It was later promoted by Hassler, Whitney, and Merrill at Princeton. Today, many formulations for the travelling salesman problem are available in the literature. The salesman problem is briefly defined here.

Given a set of cities and the cost of travel (or distance) between each possible pairs, the travelling salesman problem is to find the best possible way of visiting all the cities and returning to the starting point that minimizes the travel cost

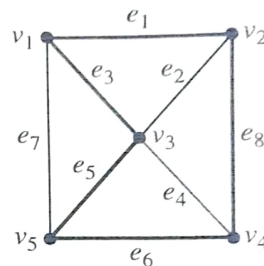


Fig. 13.35 Hamiltonian circuit

(or travel distance). In terms of graph theory, if the distance between each pair is known, the graph is a complete graph, and the travelling salesman problem is similar to finding a Hamiltonian circuit that has the minimum total distance. Given n is the number of vertices to be visited, the total number of possible Hamiltonian circuits is given by $(n-1)!/2$. Thus, there will be $(n-1)!/2$ feasible solutions for the travelling salesman problem and the solution that has the minimum total distance shall be the optimal solution.

13.11 PLANAR GRAPH

A graph is said to be planar if there exists some geometric representations of G that can be drawn on a plane such that no two of its edges intersect. A graph for which no geometric representation exists in which the edges do not intersect is called non-planar. A drawing of a geometric representation of a graph on any surface such that no edges intersect is called embedding.

For a given graph, to decide whether the graph is planar or not, we should try to find the geometric representation of the graph that can be embedded in a plane. A graph is non-planar if out of all possible geometric representations none can be embedded in a plane. The graphs given in Figs 13.36(a) and (b) are planar graphs. For the graph given in Fig. 13.36(b), a geometric representation shown in Fig. 13.36(c) exists that can be embedded in a plane; thus, the graph is planar.

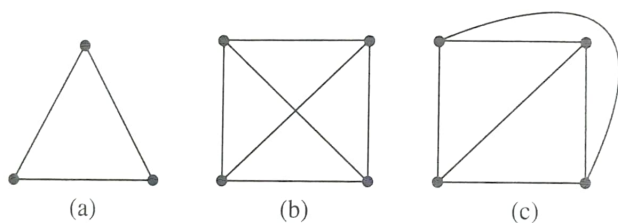


Fig. 13.36 Planar graphs (a) Planar graph with three vertices (b) Planar graph with four vertices (c) Geometric representation of graph (b)

13.11.1 Kuratowski's Two Graphs

Kuratowski, a Polish mathematician, introduced two non-planar graphs called Kuratowski's graphs. Kuratowski's first graph is a complete graph of five vertices (K_5) (Fig. 13.37) and the second graph is a regular connected graph of six vertices and nine edges ($K_{3,3}$) (Fig. 13.38).

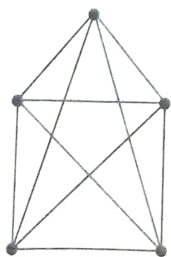


Fig. 13.37 Kuratowski's first graph (K_5)

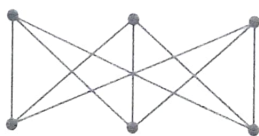


Fig. 13.38 Kuratowski's second graph ($K_{3,3}$)

The following points can be observed from the two graphs:

1. Both graphs are regular.
2. Both graphs are non-planar.
3. The removal of one edge or a vertex makes each a planar graph.
4. K_5 is a non-planar graph with the smallest number of vertices.
5. $K_{3,3}$ is a non-planar graph with the smallest number of edges.
6. Any graph isomorphic to any of the Kuratowski's graph is non-planar.

13.11.2 Region and its Degree

Planar representation of a graph divides the plane into several regions (also called windows, faces, or meshes). A region is characterized by the set of edges forming its boundary. The region of the plane lying outside a graph embedded in a plane is called an infinite, unbounded, or exterior region. In Fig. 13.39, there are four regions. The region R_4 is unbounded whereas the other three regions are bounded. The degree of any region R is the length of the closed walk that bounds the region.

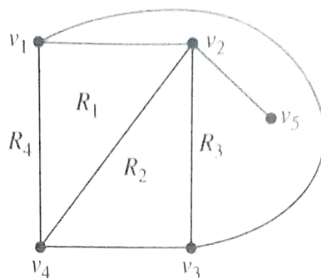


Fig. 13.39 Regions in planar graphs

In Fig. 13.39, the degrees of the regions are as follows:

$$\deg(R_1) = 3, \deg(R_2) = 3, \deg(R_3) = 5, \text{ and } \deg(R_4) = 3$$

For the region R_3 , the closed walk is $v_3 - v_1 - v_2 - v_5 - v_2 - v_3$.

Since each edge separates two regions, the sum of degrees of all regions equals twice the number of edges. For the graph given in Fig. 13.39, the total degree of all regions equals 14, which is twice the number of edges in the graph as there are seven edges in the graph.

13.11.3 Euler's Formula

A planar graph may have different plane representations. A question may arise regarding whether the number of regions resulting from each embedding is the same. The answer is yes and lies in Euler's formula, which provides the number of regions in any planar graph.

For a connected graph G with n vertices and e edges ($e > 2$), the total number of regions R is given by $e - n + 2$, that is, $R = e - n + 2$.

Check Your Progress 13.2

Check whether the following statements are true or false:

1. A path is an open non-intersecting walk.
2. Every closed walk is a circuit.
3. A component is a subgraph that is connected to itself.
4. The distance between two vertices is the length of the longest path between the two vertices.
5. The centre of a graph is the vertex having minimum eccentricity.

6. A unicursal line is an open walk that includes all edges of a graph.
7. A Hamiltonian circuit is a closed walk that traverses each edge of a graph exactly once.
8. The length of a Hamiltonian path in a graph of n vertices is $n - 1$.
9. A complete graph of four vertices is a non-planar graph.
10. $K_{3,3}$ is a planar graph.

13.12 TREE

A connected graph without any circuit is called a tree. A forest is a disjoint union of trees. From the definition of a tree, it can be observed that a tree is a simple connected graph without self-loops and parallel edges. Figure 13.40 shows trees with different numbers of vertices.

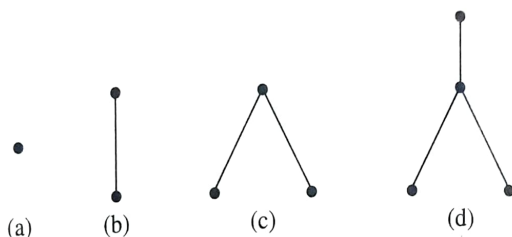


Fig. 13.40 Trees with different numbers of vertices (a) $n = 1$ (b) $n = 2$ (c) $n = 3$ (d) $n = 4$

THEOREM 13.7 Prove that there is one and only one path between every pair of vertices in a tree T .

PROOF: Since a tree is a connected graph, there exists a path between every pair of vertices. Let us assume that there are two different paths between a pair of vertices. The two different paths will form a circuit, which is a contradiction to the definition of a tree. Hence, there is one and only one path between every pair of vertices in a tree T .

THEOREM 13.8 Prove that there are $n - 1$ edges in a tree with n vertices.

PROOF: We shall use the principle of mathematical induction to prove the theorem. Let us consider a tree with only one vertex only, that is, $n = 1$. A tree with one vertex has no edge. Similarly, a tree with two vertices, that is, for $n = 2$, has only one edge. Hence, the statement is true for $n = 1$ and $n = 2$.

Let the statement be true for $n = k$; that is, a tree with k vertices has $k - 1$ edges. We shall prove the statement for $n = k + 1$. If we insert a vertex in a tree

of k vertices, then the vertex must be joined by any of the k vertices through one edge only, because a tree is a connected graph without any circuit. Thus, if the number of vertices is increased by one, then the number of edges is increased by one. Total number of vertices = $k + 1$ Total number of edges = $k - 1 + 1 = k$. This implies that the statement is true for $n = k + 1$, and hence, it is true for all natural numbers.

Thus, a tree is an undirected simple graph T that satisfies any of the following equivalent conditions:

- T is connected and has no circuits.
- T has no circuits, and a circuit is formed if any edge is added to T .
- T becomes disconnected if any edge is removed from T .
- Every two vertices of T are connected by a unique path.

13.12.1 Rooted Tree

A tree is called a rooted tree if one vertex is designated as the root in the tree and it is distinguishable from other vertices. Any vertex can be chosen as the root. In a rooted tree, the parent of a vertex is the vertex connected to it on the path to the root; every vertex except the root has a unique parent. A child of a vertex v is a vertex of which v is the parent. A leaf is a vertex without children. Figure 13.41 shows examples of rooted trees.

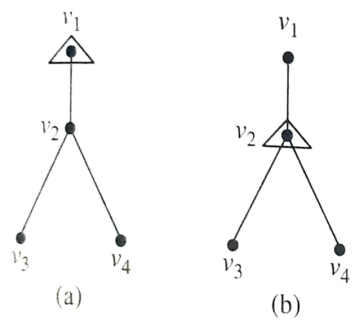


Fig. 13.41 Rooted trees (a) Rooted tree with root v_1 (b) Rooted tree with root v_2

13.12.2 Binary Tree

A tree is said to be a binary tree if each vertex has at most two children. For example, Fig. 13.42 shows different binary trees. In various algorithms in computer science, a binary tree that contains vertices of zero or two children is very useful. Thus, a particular class of binary is defined as follows:

A binary tree is said to be a *full binary tree* if there is exactly one vertex of degree two and other vertices are of degree three or one. A binary tree is called a *complete binary tree* if every level, except possibly the last, is completely filled (every vertex has left as well as right child) and all nodes are as far left as possible. Figures 13.42(a) and (c) are examples of full binary trees, whereas Fig. 12.42(b) is an example of a complete but not full binary tree.

From the definition of a binary tree, the following properties can be observed.

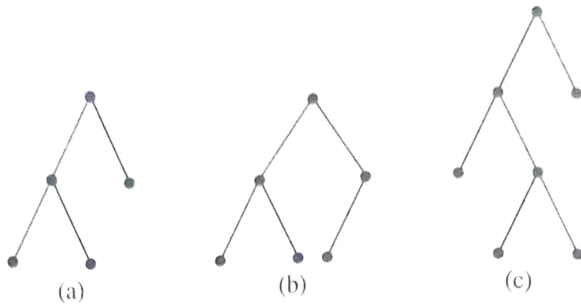


Fig. 13.42 Binary trees (a) Full and complete binary tree (b) Complete but not full binary tree (c) Full but not complete binary tree

THEOREM 13.9 The total number of vertices n in a full binary tree is always odd.

Proof: There are n vertices. The degree of one vertex is two, and that for the remaining $n - 1$ vertices is either one or three. We know that the number of vertices having an odd degree in a graph is even. Thus, $n - 1$ is an even number and hence n is an odd number.

THEOREM 13.10 The total number of pendent vertices in a full binary tree with n vertices is $\frac{n+1}{2}$.

Proof: Let the number of pendent vertices be p . In a full binary tree, the degree of one vertex is two and that for the remaining vertices is three. We know that a tree with n vertices contains $n - 1$ edges. Thus, using the equation

$$\sum_{i=1}^n \deg(v_i) = 2e, \text{ we have}$$

$$1 \cdot p + 2 \cdot 1 + 3 \cdot (n - p - 1) = 2(n - 1)$$

$$\Rightarrow p + 2 + 3n - 3p - 3 = 2n - 2$$

$$\Rightarrow -2p = -n - 1$$

$$\Rightarrow p = \frac{n+1}{2}$$

EXAMPLE 13.6

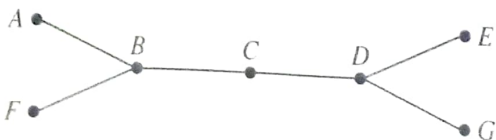


Fig. 13.43 Tree for Example 13.6

Find the centre(s) of the tree given in Fig. 13.43.

Solution: The vertex C has minimum eccentricity, that is, two. Thus, the centre of the tree is the vertex C .

13.12.3 Height of Binary Tree

A vertex v in a binary tree T is said to be at level l if its distance from the root is l . The height of a binary tree is the maximum level of any vertex in a tree. Figure 13.44 shows the level of each vertex in the given tree. We can find the minimum and maximum possible heights of a full binary tree with n vertices.

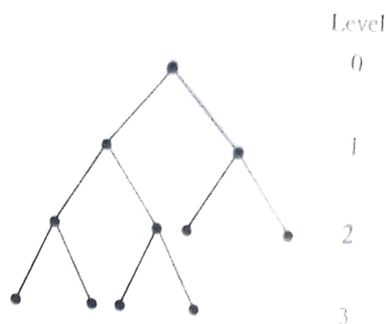


Fig. 13.44 Binary tree and level of vertices

THEOREM 13.11 The minimum height of a full binary tree with n vertices is $\lceil \log_2(n+1) - 1 \rceil$.

Proof: Let h be the height of a full binary tree with n vertices. The maximum number of vertices up to level h will be $2^0 + 2^1 + 2^2 + \dots + 2^h$. Since there are n vertices in the binary tree,

$$n \leq 2^0 + 2^1 + 2^2 + \dots + 2^h$$

$$\Rightarrow n \leq 2^0 \left(\frac{2^{h+1} - 1}{2 - 1} \right) = 2^{h+1} - 1$$

$$\Rightarrow 2^{h+1} \geq n + 1$$

$$\Rightarrow h + 1 \geq \log_2(n + 1)$$

$$\Rightarrow h \geq \log_2(n + 1) - 1$$

Hence, the minimum height of the tree is $h_{\min} = \lceil \log_2(n + 1) - 1 \rceil$.

THEOREM 13.12 The maximum height of a full binary tree with n vertices is $\frac{n-1}{2}$.

Proof: Let $h_{\max}$ be the maximum height of a full binary tree with n vertices. For the maximum height with n vertices, we must have exactly two vertices at each level except at zero level, as in Fig. 13.45.

Thus, $n = 1 + 2 + 2 + \dots + 2$ ($h_{\max}$ times)

$$\Rightarrow n = 1 + 2h_{\max}$$

$$\Rightarrow h_{\max} = \frac{n-1}{2}$$



Fig. 13.45 Example of full binary tree to find maximum height

13.12.4 Spanning Tree



Fig. 13.46 Graph $G(V, E)$

Let $G(V, E)$ be a connected graph. A tree T that is a subgraph of G is called a spanning tree if it contains all vertices of the graph G . The two spanning trees of the graph shown in Fig. 13.46 are given in Figs. 13.47(a) and (b).



Fig. 13.47 Spanning trees of graph $G(V, E)$
(a) Tree 1 (b) Tree 2

It can be observed that a spanning tree is defined only for a connected graph. This is because a tree is always connected, and for a disconnected graph, we cannot find a connected subgraph that contains all vertices of the disconnected graph. Each component of a disconnected graph, however, does have a spanning tree. Thus, a disconnected graph with k components has a spanning forest consisting of k spanning trees.

13.12.5 Branch and Chord

Let T be a spanning tree of a graph G . Then an edge $e \in T$ is called a branch and an edge $e \notin T$ is called a chord. For example, the edges e_1, e_7, e_8 , and e_3 are branches and the edges e_2, e_4, e_5 , and e_6 are chords with respect to the spanning tree T shown in Fig. 13.47(a).

It must be kept in mind that branches and chords are defined with respect to a given spanning tree; hence, if we change the spanning tree, the set of chords and branches will be changed. The set of chords and the set of branches are disjoint sets, and the union of these two sets is the set $G(E)$. Let T be a spanning tree and $\bar{T}$ be its complement in G . Then $G = T \cup \bar{T}$.

13.12.6 Rank and Nullity

It can be observed that the number of vertices n and the number of edges e are two key terms that describe a graph. Further, the number of components k is also an important factor. If $k = 1$, then the graph is connected, and if $k \geq 2$ the graph is disconnected. Since a maximum of n components are possible in a graph with n vertices, $k \leq n$ or $n - k \geq 0$. Moreover, the number of edges e in a graph cannot be less than $n - k$; that is, $e \geq n - k$ or $e - n + k \geq 0$. The three values n , e , and k are independent and are the fundamental numbers of a graph. From these three values, two other important values are derived called rank and nullity defined as

$$\text{Rank } r = n - k$$

$$\text{Nullity } \mu = e - n + k$$

The rank and nullity of a connected graph are $n - 1$ and $e - n + 1$, respectively. The two values also have significant meaning, defined as follows:

Rank of G = Number of branches in any spanning tree T (forest of spanning trees) of G

Nullity of G = Number of chords in G with respect to the tree T (forest of spanning trees)

Rank + nullity = Number of edges in G

13.12.7 Fundamental Circuits

Let us consider a connected graph G and its spanning tree T . On adding one chord to T , we get exactly one circuit. Such a circuit formed by adding a chord to a spanning tree is called a fundamental circuit. Since there are $\mu = e - n + k$ chords in a graph G , μ fundamental circuits are possible in the graph. In Fig. 13.47(a), on adding the chord e_5 to the given tree, we get the fundamental circuit $v_2e_8v_4e_3v_3e_5v_2$.

13.12.8 Finding All Spanning Trees of a Graph

For a given graph, a number of spanning trees are possible. To find all spanning trees of a graph, we can choose any arbitrary spanning tree. Write the set of branches and chords with respect to the spanning tree. On adding one chord, we get a fundamental circuit, and on removing one branch from the fundamental circuit, we get another spanning tree. Proceeding in the same way, we can find all spanning trees in a graph.

For example, to find all spanning trees for the graph in Fig. 13.46, we can choose one spanning tree T_1 given in Fig. 13.47(a). The set of branches with respect to the spanning tree T_1 is $\{e_1, e_7, e_8, e_3\}$ and the set of chords is $\{e_2, e_4, e_5, e_6\}$. Now on adding the chord e_2 , we get the fundamental circuit $v_5e_7v_2e_8v_4e_2v_5$. There are two branches in the fundamental circuit. On deleting the edges e_7 and e_8 , we get two spanning trees T_2 and T_3 , respectively (Fig. 13.48). Proceeding in the same way we get all spanning trees.

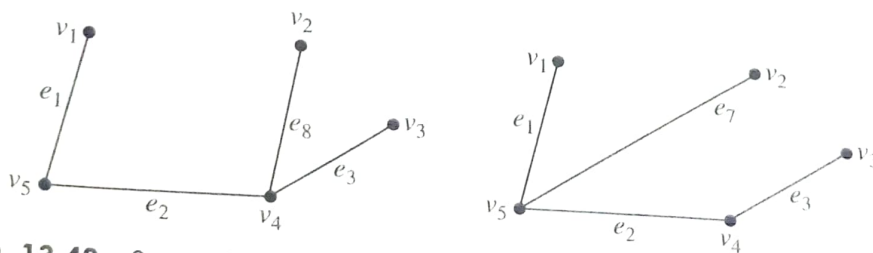


Fig. 13.48 Spanning trees T_2 and T_3 of the graph given in Fig. 13.46

13.12.9 Spanning Trees in a Weighted Graph

In a weighted graph G , a real number (weight) is associated with each edge of G . The weight of a spanning tree T of the graph G is defined as the sum of the weights of the branches in T . Different spanning trees will have different

13.17 TRAVERSAL OF GRAPHS

Traversal of a graph is to visit all vertices of the graph in a specific order. There are two standard ways of traversing a graph, breadth-first search and depth-first search. For traversing purpose, we define three states for a vertices, namely ready state, waiting state, and processed state. The graph thus obtained using these algorithms is known as a breadth-first search tree or depth-first search tree, which is itself a spanning tree of the graph.

13.17.1 Breadth-first Search

In breadth-first search, the general idea is to start at a designated vertex and then explore the neighbours of the original vertex. Once all neighbours have been explored, their neighbours are explored. This process repeats until all vertices of the entire graph have been visited. However, we need to ensure that no vertex is

processed more than once. Breadth-first search is accomplished by using a queue to hold the vertices that are waiting to be processed. Here, one should remember that a queue is a data structure where entries are added to the tail (also called the rear) and removed from the front. The following is the algorithm for breadth-first search.

Algorithm BFS

1. Initialize all vertices to ready state.
2. Move the starting vertex to the queue and change its status to waiting state.
- Repeat steps 4 and 5 until the queue is empty.
3. Remove front vertex v from the queue, process it, and change its status to processed state.
4. Add to the rear of the queue all the neighbours of the vertex v that are in ready state and change their status to waiting state.
5. Exit.

EXAMPLE 13.12

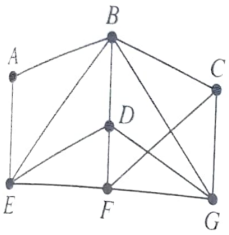


Fig. 13.67 Graph for Example 13.12

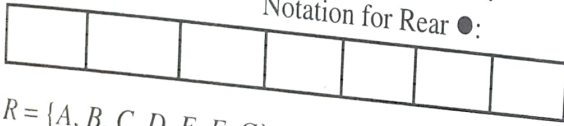
Apply a breadth-search algorithm to explore all the vertices starting from the vertex A of the graph given in Fig. 13.67 and find the breadth-first search tree.

Solution: Let R , W , and P be the set of vertices in the ready state, waiting state, and processed state, respectively, and P_E be the set of processed edges.

Step 1 Queue

Notation for Front ○:

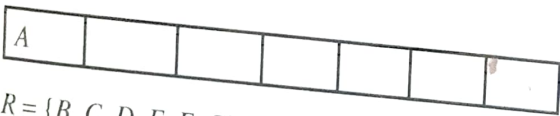
Notation for Rear ●:



$R = \{A, B, C, D, E, F, G\}$, $W = \{\}$, $P = \{\}$, $P_E = \{\}$

Step 2 Queue

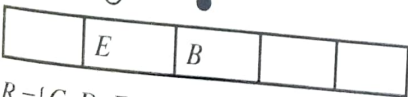
○ ●



$R = \{B, C, D, E, F, G\}$, $W = \{A\}$, $P = \{\}$, $P_E = \{\}$

Step 3 Queue (Process and remove A and add its neighbours E, B from the rear.)

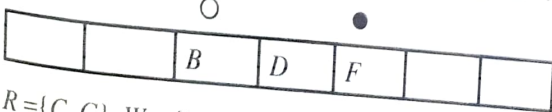
○ ●



$R = \{C, D, F, G\}$, $W = \{E, B\}$, $P = \{A\}$, $P_E = \{\}$

Step 4 Queue (Process and remove E and add its neighbours D, F from the rear.)

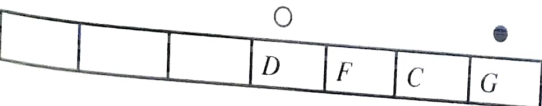
○ ●



$R = \{C, G\}$, $W = \{B, D, F\}$, $P = \{A, E\}$, $P_E = \{(A, E)\}$

Step 5 Queue (Process and remove B and add its neighbours C, G from the rear.)

○ ●



$R = \{ \}, W = \{ D, F, C, G \}, P = \{ A, E, B \}, P_E = \{ (A, E), (A, B) \}$

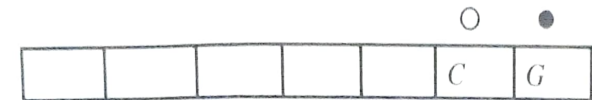
All neighbours of A have been traversed. Now we will look for the neighbours of E.

Step 6 Queue (process and remove D. No neighbour of D can be added as R is empty.)



$R = \{ \}, W = \{ F, C, G \}, P = \{ A, E, B, D \}, P_E = \{ (A, E), (A, B), (E, D) \}$

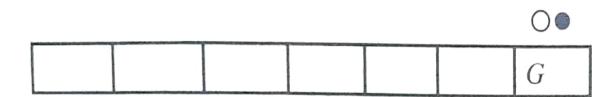
Step 7 Queue (Process and remove F. No neighbour of F can be added as R is empty.)



$R = \{ \}, W = \{ C, G \}, P = \{ A, E, B, D, F \}, P_E = \{ (A, E), (A, B), (E, D), (E, F) \}$

All neighbours of E have been traversed. Now we will look for the neighbours of B.

Step 8 Queue (Process and remove C. No neighbour of C can be added as R is empty.)



$R = \{ \}, W = \{ G \}, P = \{ A, E, B, D, F, C \}$

$P_E = \{ (A, E), (A, B), (E, D), (E, F), (B, C) \}$

Step 9 Queue (Process and remove G.)

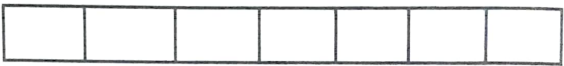
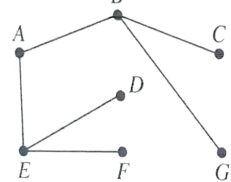


Fig. 13.68 Breadth-first search tree for Example 13.12

$R = \{ \}, W = \{ \}, P = \{ A, E, B, D, F, C, G \}$

$P_E = \{ (A, E), (A, B), (E, D), (E, F), (B, C), (B, G) \}$

Since the queue is empty, the set of processed edges is $\{ (A, E), (A, B), (E, D), (E, F), (B, C), (B, G) \}$, which traverses all vertices of the graph. The breadth-first search tree is shown in Fig. 13.68.

13.17.2 Depth-first Search

In the depth-first search algorithm, the general idea is to start at a designated vertex and go deeper in the graph. This algorithm explores a path in the graph as long as possible by selecting one neighbour at a time. When it is not possible to go forward, then the algorithm backtracks one level and again tries to go deeper. This process is repeated until all vertices of the entire graph have been visited. Depth-first search is accomplished by using a stack to hold the vertices that are waiting to be processed. Here, one should remember that a stack is a data structure where entries are added and removed only at the top. These two operations—adding to and removing from the stack—are known as push and pop, respectively. The algorithm is as follows:

- 1. Initialize all vertices to ready state.
- 2. Push the starting vertex to stack and change its status to waiting state.
- 3. Repeat steps 4 and 5 until the stack is empty.
- 4. Pop the top vertex v from the stack, process it, and change its status to processed state.

5. Push onto the stack all the neighbours of v that are in ready state and change their status to waiting state.
6. Exit.

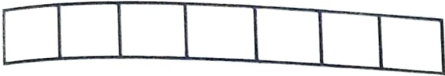
EXAMPLE 13.13

Apply a depth-search algorithm to explore all the vertices starting from the vertex A of the graph given in Fig. 13.67.

Solution:

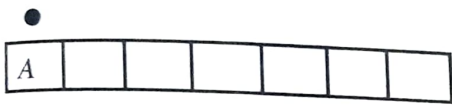
Step 1 Stack

Notation for Top: ●



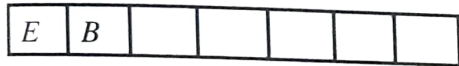
$R = \{A, B, C, D, E, F, G\}$, $W = \{\}$, $P = \{\}$, $P_E = \{\}$

Step 2 Stack



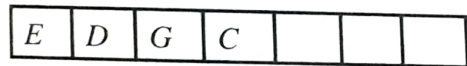
$R = \{B, C, D, E, F, G\}$, $W = \{A\}$, $P = \{\}$, $P_E = \{\}$

Step 3 Stack (Process and remove A and push its neighbours E, B onto the stack.)



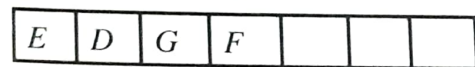
$R = \{C, D, F, G\}$, $W = \{E, B\}$, $P = \{A\}$, $P_E = \{\}$

Step 4 Stack (Process and remove B and push its neighbours D, G , and C onto the stack.)



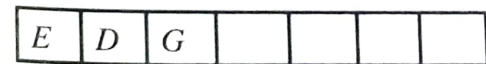
$R = \{F\}$, $W = \{E, D, G, C\}$, $P = \{A, B\}$, $P_E = \{(A, B)\}$

Step 5 Stack (Process and remove C and push its neighbour F onto the stack.)



$R = \{\}$, $W = \{E, D, F, G\}$, $P = \{A, B, C\}$, $P_E = \{(A, B), (B, C)\}$

Step 6 Stack (Process and remove F . No neighbor can be pushed onto the stack as R is empty.)



$R = \{\}$, $W = \{E, D, G\}$, $P = \{A, B, C, F\}$, $P_E = \{(A, B), (B, C), (C, F)\}$

Step 7 Stack (Process and remove G . No neighbour can be pushed onto the stack as R is empty)



$R = \{\}$, $W = \{E, D\}$, $P = \{A, B, C, G, F\}$, $P_E = \{(A, B), (B, C), (C, F), (B, G)\}$